

A project report on

IOT BASED HOME AUTOMATION SYSTEM

Submitted in partial fulfilment for the award of the degree of

B.Sc. (Computer Science)

2nd REVIEW

by

Vivekananthan S (21BCS0078)

Dhanush K(21BCS0166)

Under the Guidance of

Prof. SenthilKumar N



VIT[®]

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

**SCHOOL OF COMPUTER SCIENCE ENGINEERING AND
INFORMATION SYSTEMS**

May 2024

Proposed Methodology

This document outlines a comprehensive methodology for developing an IoT-based smart home system, emphasizing user experience and future scalability. We'll leverage Flutter for a user-friendly mobile application and ESP32 for device control.

1. Self-Directed Learning and Feature Exploration:

- As this project is primarily for self-directed learning, explore various resources (online tutorials, documentation) to understand the fundamentals of Flutter and ESP32 development, focusing on functionalities relevant to building a smart home system (e.g., sensor data acquisition, control commands).
- Identify core features we would like to implement for this learning project (e.g., light/fan control, basic temperature monitoring). This allows you to focus on specific functionalities while exploring core concepts.

2. System Architecture with Security in Mind:

- Design a secure system architecture with clear separation between the mobile app (presentation), ESP32 (data acquisition and control), and communication protocol, prioritizing data security throughout.
- Implement robust authentication and authorization mechanisms for both the mobile app and ESP32 communication (e.g., secure password protocols, token-based authentication).
- Consider data encryption during transmission (especially for Wi-Fi Direct) to protect sensitive information.

3. User-Centered Design and Prototyping:

- Develop detailed user interface (UI) mockups incorporating user feedback and design best practices. Focus on intuitive controls, clear visualizations of sensor data, and a visually appealing aesthetic.

4. Continuous Integration/Delivery (CI/CD):

- Adopt an agile development methodology with iterative development cycles for both the mobile app and ESP32 firmware. This allows for continuous adaptation based on user feedback and changing requirements.
- Set up a CI/CD pipeline to automate testing, building, and deployment processes, ensuring efficient development and rapid updates.

5. Comprehensive Development:

Mobile App Development (Flutter):

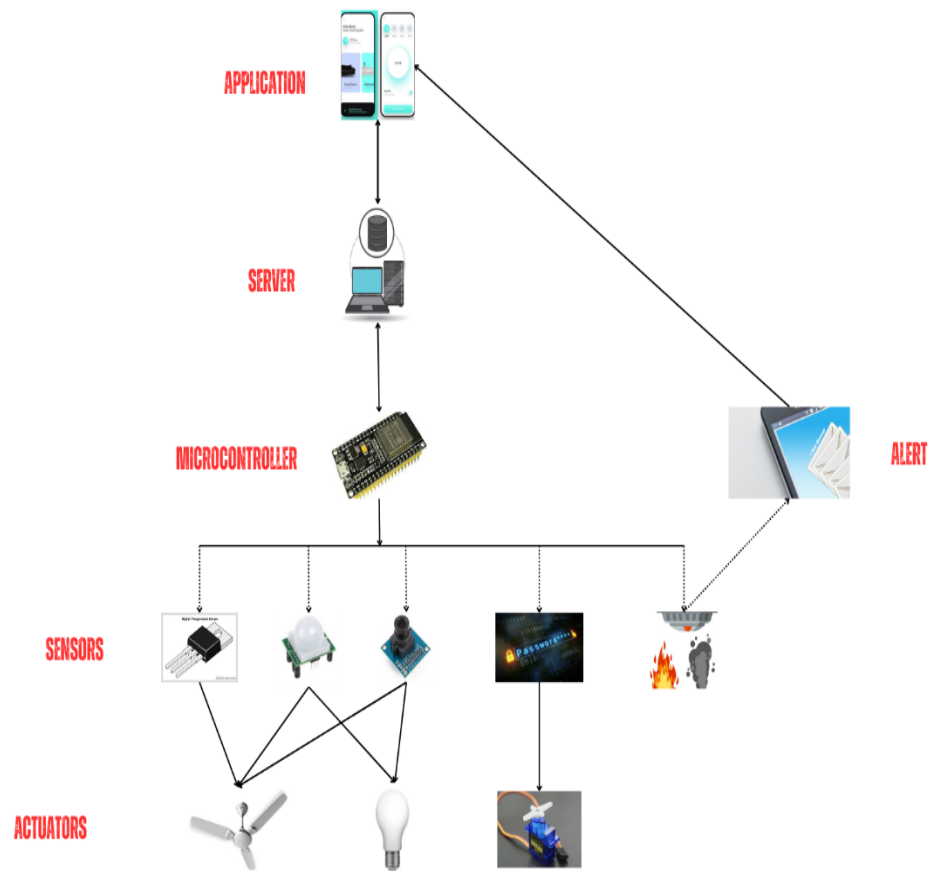
- Implement the user interface based on the user-tested prototypes using Flutter widgets.
- Integrate communication libraries for secure data exchange with the ESP32 (e.g., Wi-Fi Direct or MQTT client libraries with encryption).
- Develop functionalities for sending control commands (lights, fans, thermostats – if applicable) and receiving sensor data (temperature, motion, air quality – if applicable) in real-time.

- Implement user authentication, authorization, and secure data storage (consider options like local storage or cloud databases with encryption).
- Explore integrating with cloud platforms (e.g., Firebase) for remote access, data logging, and potential future functionalities.

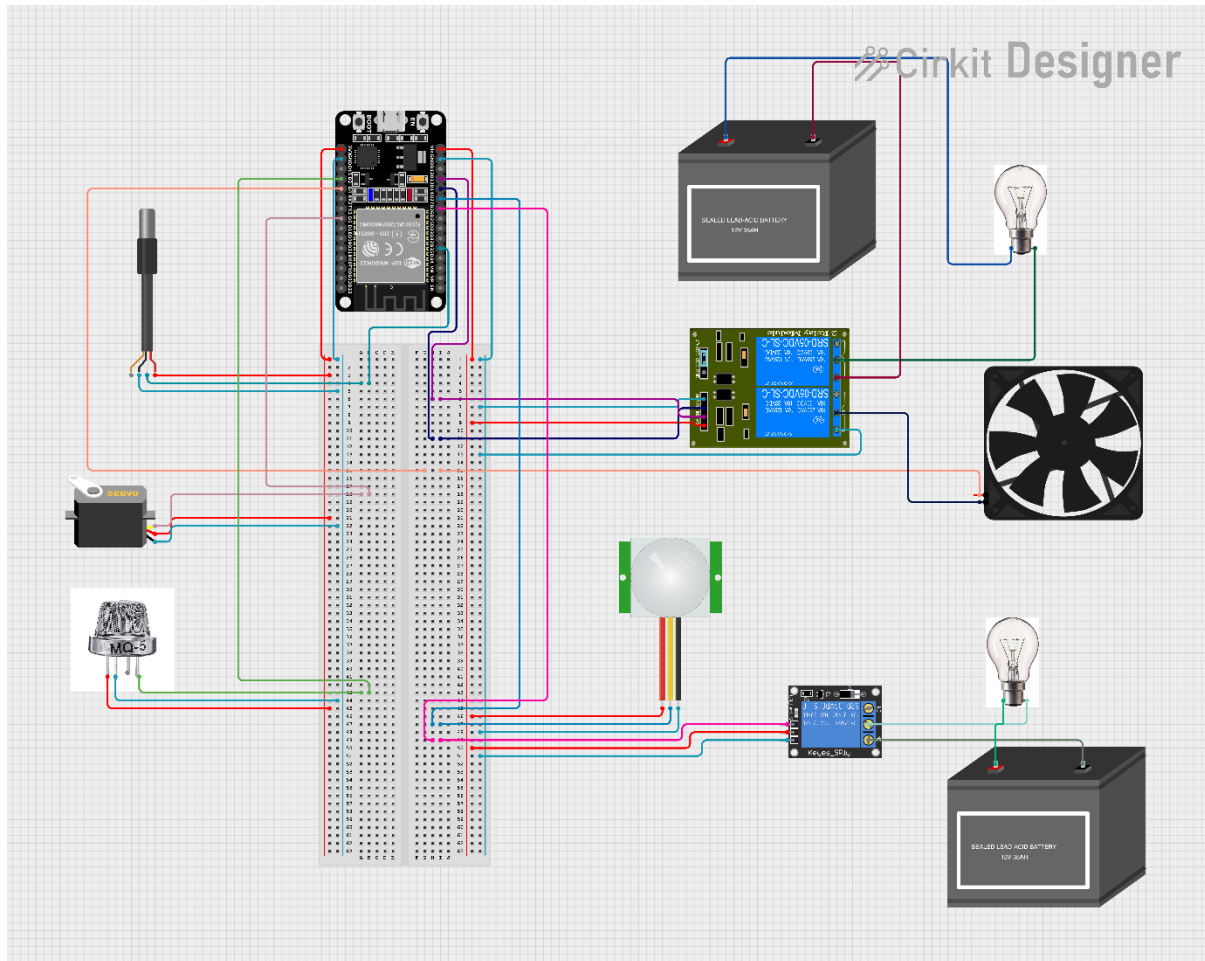
ESP32 Firmware Development:

- Program the ESP32 to securely connect to the chosen communication protocol (Wi-Fi Direct or MQTT Broker).
- Develop code to read sensor data (temperature, motion, air quality – if applicable) and control actuators (lights, fans, thermostats – if applicable) based on encrypted commands received from the mobile app.
- Implement security measures (e.g., secure boot, user authentication) to prevent unauthorized access to the ESP32 device.
- Consider incorporating over-the-air (OTA) firmware updates for easy bug fixes and feature rollouts.

Architecture



Complete Design



Module Description

1. ESP32 Development Board:

- The ESP32 remains the core microcontroller board. It handles:
- Wi-Fi connectivity (or Wi-Fi Direct) for communication with the mobile app.
- Reading sensor data from connected sensors.
- Controlling actuators based on app commands.
- Processing data and communication using the chosen protocol.

2. Sensors:

- Temperature Sensor (DS18B20): Measures ambient temperature for functions like smart thermostat control or monitoring comfort levels.
- Motion Sensor (PIR sensor): Detects motion for security purposes (triggering alarms) or automated lighting control (turning on lights when someone enters a room).

- MQ-5 Gas Sensor: Detects the presence of various gases for air quality monitoring (important for potential gas leaks).

3. Actuators:

- Servo Motor: Offers precise control over angular movement. It can be used for:
- Controlling blinds or curtains based on time of day or temperature.
- Adjusting the position of a camera for security monitoring.
- Relay Module: Acts like a switch controlled by the ESP32. It can be used to turn on/off various appliances based on sensor data or user commands (e.g., lights, fans).

4. Communication Protocol (Wi-Fi Direct or MQTT):

- Establishes a connection between the ESP32 and the mobile app.
- MQTT (Message Queue Telemetry Transport): More flexible and scalable, using a central broker for message exchange but requiring additional configuration.

5. Flutter Mobile App:

- Developed using Flutter, the app provides a user interface for interacting with the smart home system. It will:
- Connect to the ESP32 via the chosen protocol (MQTT).
- Display sensor data (temperature, motion detection – if using motion sensor, air quality – if using MQ-5).
- Offer controls for actuators (e.g., turning on/off lights, adjusting fan speed, controlling servo motor position – if applicable).
- Potentially allow for setting schedules or automation rules for smart home devices.

Additional Considerations:

- Breadboard and Jumper Wires: These help connect all the electronic components for prototyping and testing.
- Power Supply: Provides power to the ESP32 board and any additional components that require it (e.g., sensor modules, servo motor).

Implementation

login_screen.dart:

```
import 'package:flutter/material.dart';

class LoginScreen extends StatelessWidget {
  const LoginScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Container(
        decoration: BoxDecoration(
          gradient: LinearGradient(
            begin: Alignment.topCenter,
            end: Alignment.bottomCenter,
            colors: [
              Colors.lightBlue[100]!,
              Colors.pink[100]!,
            ],
          ),
        ),
      ),
      child: Center(
        child: Padding(
          padding: const EdgeInsets.all(20.0),
          child: SingleChildScrollView(
            child: Column(
              mainAxisAlignment: MainAxisAlignment.center,
              children: <Widget>[
                const Text(
                  'Welcome Back!',
                  style: TextStyle(
                    fontSize: 30,
                    fontWeight: FontWeight.bold,
                  ),
                ),
                const SizedBox(height: 30.0),
                TextField(
                  decoration: InputDecoration(
                    labelText: 'Email',
                    prefixIcon: const Icon(Icons.email),
                    filled: true,
                    fillColor: Colors.white,
                    border: OutlineInputBorder(
                      borderRadius: BorderRadius.circular(10),
                      borderSide: BorderSide.none,
                    ),
                  ),
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

```

    ),
  ),
),
const SizedBox(height: 20.0),
TextField(
  obscureText: true,
  decoration: InputDecoration(
    labelText: 'Password',
    prefixIcon: const Icon(Icons.lock),
    filled: true,
    fillColor: Colors.white,
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(10),
      borderSide: BorderSide.none,
    ),
  ),
),
const SizedBox(height: 20.0),
ElevatedButton(
  onPressed: () {
    // Implement login functionality
    Navigator.pushNamed(context, '/home');
  },
  style: ElevatedButton.styleFrom(
    backgroundColor: Colors.blue,
    padding: const EdgeInsets.symmetric(horizontal: 50,
vertical: 15),
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(10),
    ),
  ),
  child: const Text(
    'Login',
    style: TextStyle(fontSize: 18),
  ),
),
TextButton(
  onPressed: () {
    // Navigate to registration screen

    Navigator.pushNamed(context, '/register');
  },
  style: TextButton.styleFrom(
    padding: const EdgeInsets.all(10),
  ),
  child: const Text(
    'Create an account',

```

```

        style: TextStyle(fontSize: 16),
      ),
    ),
  ],
),
),
),
),
),
),
);
}
}

```

Register_screen.dart:

```

import 'package:flutter/material.dart';

class RegisterScreen extends StatelessWidget {
  const RegisterScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Register'),
      ),
      body: Container(
        decoration: BoxDecoration(
          gradient: LinearGradient(
            begin: Alignment.topCenter,
            end: Alignment.bottomCenter,
            colors: [
              Colors.lightBlue[100]!,
              Colors.pink[100]!,
            ],
          ),
        ),
        child: Center(
          child: Padding(
            padding: const EdgeInsets.all(20.0),
            child: SingleChildScrollView(
              child: Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: <Widget>[
                  const Text(
                    'Create an Account',
                    style: TextStyle(

```

```

        fontSize: 30,
        fontWeight: FontWeight.bold,
      ),
    ),
    const SizedBox(height: 30.0),
    TextField(
      decoration: InputDecoration(
        labelText: 'Name',
        prefixIcon: const Icon(Icons.person),
        filled: true,
        fillColor: Colors.white,
        border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(10),
          borderSide: BorderSide.none,
        ),
      ),
    ),
    const SizedBox(height: 20.0),
    TextField(
      decoration: InputDecoration(
        labelText: 'Email',
        prefixIcon: const Icon(Icons.email),
        filled: true,
        fillColor: Colors.white,
        border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(10),
          borderSide: BorderSide.none,
        ),
      ),
    ),
    const SizedBox(height: 20.0),
    TextField(
      obscureText: true,
      decoration: InputDecoration(
        labelText: 'Password',
        prefixIcon: const Icon(Icons.lock),
        filled: true,
        fillColor: Colors.white,
        border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(10),
          borderSide: BorderSide.none,
        ),
      ),
    ),
    const SizedBox(height: 20.0),
    ElevatedButton(
      onPressed: () {
        // Implement registration functionality

```

```

        },
        style: ElevatedButton.styleFrom(
          backgroundColor: Colors.blue,
          padding: const EdgeInsets.symmetric(horizontal: 50,
vertical: 15),
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(10),
          ),
        ),
        child: const Text(
          'Register',
          style: TextStyle(fontSize: 18),
        ),
      ),
    ],
  ),
),
),
),
),
);
}
}

```

Home_screen.dart:

```

import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';

class HomePage extends StatefulWidget {
  const HomePage({super.key});

  @override
  State<HomePage> createState() => _HomePageState();
}

class _HomePageState extends State<HomePage> {
  // Placeholder variables for future functionality
  bool _livingRoomLight1 = false;
  bool _livingRoomLight2 = false;
  bool _bedroomLight1 = false;
  bool _bedroomLight2 = false;
  bool _livingRoomFan = false;
  bool _bedroomFan = false;
  double _livingRoomTemp = 22.0;
  double _bedroomTemp = 20.0;
  String _doorPassword = ""; // Replace with your actual password

```

```

    // Placeholder comments to indicate functions for future backend
implementation
    // void _toggleLivingRoomLight1() { ... } // Implement logic to control
light 1

void _updateTemperatures(double livingRoomTemp, double bedroomTemp) {
    setState(() {
        _livingRoomTemp = livingRoomTemp;
        _bedroomTemp = bedroomTemp;
    });
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(
            title: Text(
                "My Smart Home",
                style: GoogleFonts.montserrat(
                    fontSize: 20, color: Colors.white), // Adjust as needed
            ),
        ),
        body: SingleChildScrollView(
            child: Column(
                children: [
                    _buildRoomSection(
                        "Living Room",
                        _livingRoomLight1,
                        // Replace with actual function calls when implemented in
backend
                        () => setState(() => _livingRoomLight1 = !_livingRoomLight1),
                        _livingRoomLight2,
                        () => setState(() => _livingRoomLight2 = !_livingRoomLight2),
                        _livingRoomFan,
                        () => setState(() => _livingRoomFan = !_livingRoomFan),
                        _livingRoomTemp,
                    ),
                    _buildRoomSection(
                        "Bedroom",
                        _bedroomLight1,
                        // Replace placeholders with actual function calls
                        () => setState(() => _bedroomLight1 = !_bedroomLight1),
                        _bedroomLight2,
                        () => setState(() => _bedroomLight2 = !_bedroomLight2),
                        _bedroomFan,
                        () => setState(() => _bedroomFan = !_bedroomFan),
                        _bedroomTemp,
                    ),
                ],
            ),
        ),
    );
}

```

```

        _buildDoorControlSection(),
      ],
    ),
  ),
);
}

Widget _buildRoomSection(
  String roomName,
  bool light1State,
  Function toggleLight1,
  bool light2State,
  Function toggleLight2,
  bool fanState,
  Function toggleFan,
  double temperature) {
  return Container(
    padding: const EdgeInsets.all(20),
    margin: const EdgeInsets.symmetric(vertical: 10),
    decoration: BoxDecoration(
      borderRadius: BorderRadius.circular(10),
      color: Colors.grey[200],
    ),
    child: Column(
      children: [
        Text(
          roomName,
          style: GoogleFonts.montserrat(
            fontSize: 18,
            fontWeight: FontWeight.bold,
          ),
        ),
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            IconButton(
              onPressed: toggleLight1,
              icon: Icon(
                light1State ? Icons.lightbulb : Icons.lightbulb_outline,
                color: Colors.amber,
              ),
            ),
            IconButton(
              onPressed: toggleLight2,
              icon: Icon(
                light2State ? Icons.lightbulb : Icons.lightbulb_outline,
                color: Colors.amber,
              ),
            ),
          ],
        ),
      ],
    ),
  );
}

```

```

        ),
        IconButton(
            onPressed: toggleFan,
            icon: Icon(
                fanState ? Icons.air_conditioner :
Icons.air_conditioner_outlined,
                color: Colors.teal,
            ),
        ),
    ],
),
Row(
    mainAxisAlignment: MainAxisAlignment.start,
    children: [
        Text(
            "Temperature: ${temperature.toStringAsFixed(1)}°C",
        ),
    ],
),
],
),
);
}

// ... previous code ...

Widget _buildDoorControlSection() {
    return Container(
        padding: const EdgeInsets.all(20),
        margin: const EdgeInsets.symmetric(vertical: 10),
        decoration: BoxDecoration(
            borderRadius: BorderRadius.circular(10),
            color: Colors.grey[200],
        ),
        child: Column(
            children: [
                Text(
                    "Door",
                    style: GoogleFonts.montserrat(
                        fontSize: 18,
                        fontWeight: FontWeight.bold,
                    ),
                ),
                TextField(
                    obscureText: true, // Password field hides input
                    decoration: const InputDecoration(
                        hintText: "Enter Password",
                    ),
                    onChanged: (value) => setState(() => _doorPassword = value),
                ),
            ],
        ),
    );
}

```

```

    ),
    ElevatedButton(
      onPressed: _doorPassword.isEmpty ? null : _handleOpenDoor, //
Disable if password empty
      child: const Text("Open Door"),
    ),
  ],
),
);
}

void _handleOpenDoor() {
  // Placeholder for backend communication (ESP32 password verification)
  // Replace with logic to send password to ESP32 and handle response
(success/failure)
  if (_doorPassword == "") {
    print("Door opened successfully!");
  } else {
    print("Wrong password!");
  }
}
}
}

```

Main.dart:

```

import 'package:flutter/material.dart';

import 'login_screen.dart';
import 'main_screen.dart';
import 'register_screen.dart';
import 'home_page.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

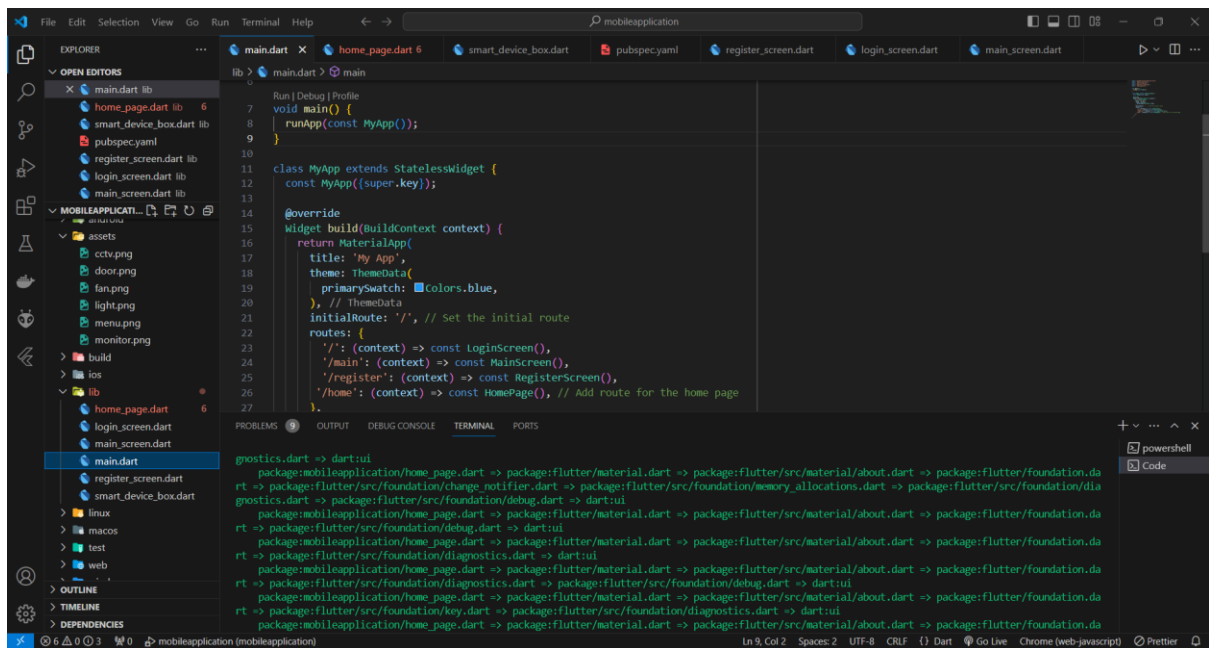
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'My App',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      initialRoute: '/', // Set the initial route
      routes: {
        '/': (context) => const LoginScreen(),

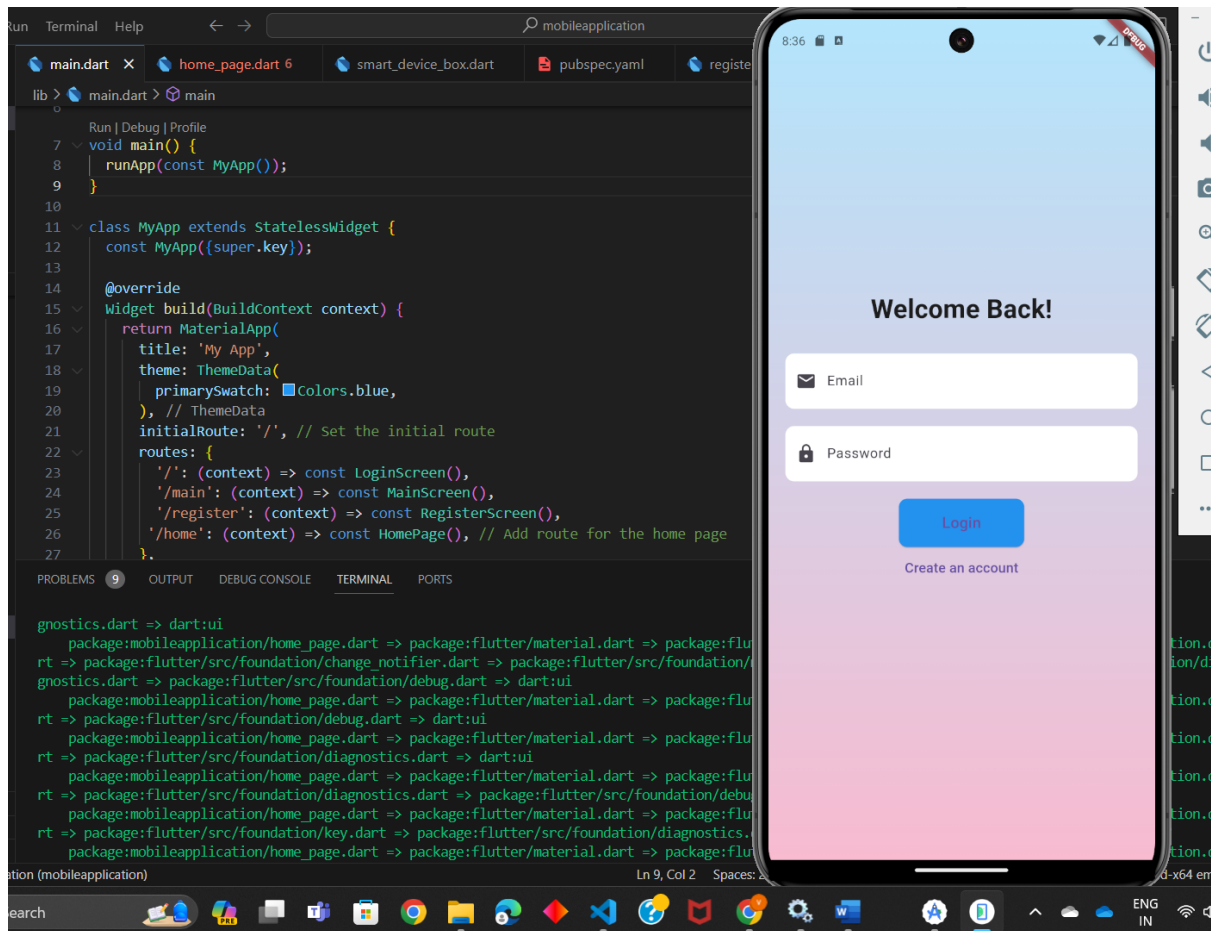
```

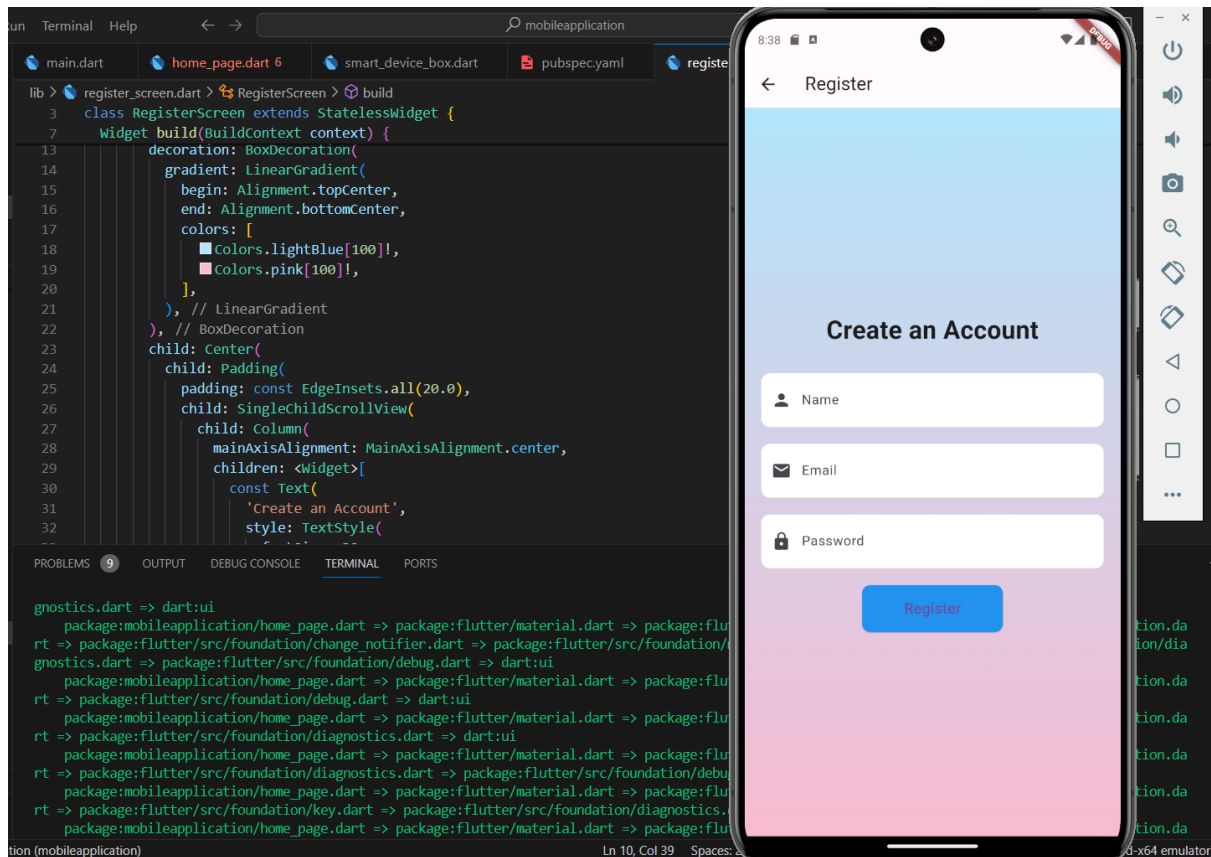
```

    '/main': (context) => const MainScreen(),
    '/register': (context) => const RegisterScreen(),
    '/home': (context) => const HomePage(), // Add route for the home page
  },
);
}
}

```







Testing

Note: The message printed are Printed in the Serial Monitor.

Test case no	Sensor Testing	Expected Output	Actual Output	Result
TC_1	Temperature in farenheit	66- 78 F	76 F	Pass
TC_2	Smoke sensor	LPG gas detected	LPG gas detected	Pass
TC_3	Servo motor	90 degree rotated	90 degree rotated	Pass
TC_4	Relay	On	ON	Pass
TC_5	Blink Led	HIGH: ON	HIGH: ON	Pass
TC_6	Fan	Spinning	Spinning	Pass
TC_7	WIFI connection	WIFI Connected	WIFI Not Connected	Fail
TC_8	WIFI Connection	WIFI Connected	WIFI Connected	Pass
TC_9	Motion detection	HIGH	LOW	Fail
TC_10	Motion detection	HIGH	HIGH	Pass

The two Fail occur because of the improper credentials provided in the WIFI Password Followed by Logical mistake in Motion sensor code.

Performance metrics

Mobile App Performance:

- **App Launch Time:** Measures the time it takes for the app to launch on the mobile device after being tapped. A faster launch time improves user experience.
- **Responsiveness:** Measures how quickly the app responds to user interactions (e.g., button presses, swipes). A responsive app feels smooth and fluid.
- **Memory Usage:** Monitors the amount of memory the app consumes while running. Lower memory usage ensures the app runs efficiently and doesn't drain the device's battery.
- **Network Traffic:** Tracks the amount of data the app sends and receives over the network. This is important for users on cellular data plans.

ESP32 Performance:

- **Power Consumption:** Measures the amount of power the ESP32 board consumes while operating. Lower power consumption is crucial for battery-powered applications or to extend the lifespan of a device plugged into an outlet.
- **Sensor Reading Latency:** Measures the time it takes for the ESP32 to read data from a connected sensor. Lower latency is desirable for applications requiring real-time data (e.g., motion detection for security).
- **Data Processing Time:** Measures the time it takes for the ESP32 to process sensor data and prepare it for transmission to the mobile app. Faster processing allows for quicker response times and handling of complex calculations (if applicable).
- **Wi-Fi/MQTT Connection Stability:** Tracks the frequency of connection drops or interruptions between the ESP32 and the chosen communication network (Wi-Fi or MQTT broker). A stable connection ensures reliable data transmission.

System-Level Performance:

- **End-to-End Latency:** Measures the total time it takes for a user action in the mobile app to trigger a response from the ESP32 and reflect in the app (e.g., turning on a light). Lower latency enhances user experience by providing a sense of immediate responsiveness.
- **Scalability:** Evaluates the system's ability to handle an increasing number of devices or users without significant performance degradation. Consider if your system can accommodate future expansion of your smart home setup.
- **Security:** Measures the system's resistance to unauthorized access, data breaches, or malicious attacks. This is crucial for protecting user privacy and preventing control of smart home devices by unauthorized users.

Reference

https://youtu.be/A5CB4t9sukM?si=IR0rB0ZJNe_sIGvR

<https://www.youtube.com/@EngineeringMindset/playlists>

https://youtu.be/TSwgZn2FKUw?si=rr6udL3_eFkGgOr

<https://youtu.be/CzRQ9mnmh44?si=DtkyWdg44sT8Lcyo>